

HANSEL_MESH 발표용 코드 설명서

통신부 / 구동부 / 영상 품질 기반 자동분리 / 현장 실행 절차

Raspberry Pi BATMAN - adv Mesh Rescue Robot Project

발표 핵심: 영상이 끊기기 전에 감속하고, 위험이 지속되면 node를 순차 분리해 통신 경로를 유지한다.

작성일: 2026 - 06 - 07 대상: 팀원 발표, 코드 리뷰, 현장 테스트 전 설명

1. 프로젝트 한눈 요약

HANSEL_MESH는 붕괴 현장 내부로 진입하는 다유닛 로봇이 통신 가능 거리를 늘리기 위한 시스템이다. 구조자는 바깥에서 노트북으로 조종하고, head에 달린 카메라 영상을 보며 전진한다. 통신 품질이 나빠질 때 node를 순차적으로 분리해서, base와 head 사이의 무선 경로를 relay로 유지한다.

핵심 목표:

- 조종 명령과 카메라 영상이 base, node, head 사이에서 end - to - end로 전달된다.
- base는 명령 내용을 해석하지 않고 유선망과 mesh망을 연결하는 gateway 역할만 한다.
- node1, node2는 애플리케이션 relay가 아니라 BATMAN - adv Layer 2 relay 역할을 한다.
- head만 조향하고 node들은 head가 만든 경로를 직진 또는 저속 추종한다.
- 영상이 끊기면 원격 조종이 불가능하므로 자동분리 기준은 RSSI보다 영상 품질을 우선한다.

전체 흐름:

```

구조자 노트북 192.168.60.2
|
| 유선 LAN
|
base eth0 192.168.60.1
base bat0 192.168.50.1
|
| BATMAN-adv mesh over wlan0 IBSS
|
node2 bat0 192.168.50.12
|
node1 bat0 192.168.50.11
|
head bat0 192.168.50.10
|
Camera + motor + servo
  
```

2. 코드 구조

```

configs/
  base.env, head.env, node1.env, node2.env, node3.env

scripts/
  start_mesh.sh
  stop_mesh.sh
  check_mesh.sh
  start_role_network.sh
  enable_mesh_autostart.sh
  setup_base_gateway.sh
  setup_laptop_mesh_routes.sh
  setup_mesh_route_to_laptop.sh
  start_camera_stream.sh
  restart_camera_profile.sh
  receive_camera_stream.sh
  deploy_from_base.sh

controller/
  mesh_control_client.py
  quality_supervisor.py

robot/
  mesh_control_server.py
  motor_driver.py

monitor/
  video_probe.py
  metrics_agent.py
  dashboard.py
  web/index.html
  
```

핵심 파일 역할:

파일	실행 위치	역할
scripts/start_mesh.sh	각 Pi	wlan0를 IBSS/mesh 링크로 만들고 bat0를 생성
scripts/start_role_network.sh	각 Pi/systemd	role별 mesh + route 자동 시작
controller/mesh_control_client.py	노트북	키보드 조종, 품질 감시, 자동분리 명령 송신
controller/quality_supervisor.py	노트북	영상 품질, ping, TQ로 GOOD/WARN/DANGER 판단
robot/mesh_control_server.py	head/node	UDP 명령 수신, 모터/서보 제어 호출
robot/motor_driver.py	head/node	GPIO, 엔코더, PID, DC 모터, servo 제어
scripts/start_camera_stream.sh	head	rpicap - vid 영상을 RTP/H.264 UDP로 송신
monitor/video_probe.py	노트북	영상 수신 + FPS/error/drop 측정
monitor/metrics_agent.py	각 Pi	RSSI, BATMAN TQ, RTT 수집
monitor/dashboard.py	노트북	통신/영상 품질 dashboard

3. 네트워크 구현

3.1 IP 설계

장치	인터페이스	IP
laptop	유선 LAN	192.168.60.2/24
base	eth0	192.168.60.1/24
base	bat0	192.168.50.1/24
head	bat0	192.168.50.10/24
node1	bat0	192.168.50.11/24
node2	bat0	192.168.50.12/24
node3	bat0	192.168.50.13/24

노트북과 base 사이의 유선망은 192.168.60.0/24이고, 로봇 mesh망은 192.168.50.0/24다.

3.2 BATMAN-adv를 쓰는 이유

BATMAN - adv는 Linux kernel 안에서 동작하는 Layer 2 mesh protocol이다. 일반 IP routing처럼 사용자가 hop을 직접 지정하지 않아도, 각 node가 자기 주변 neighbor를 보고 목적지까지의 next - hop을 선택한다.

이 프로젝트에서의 의미:

- 모든 Pi가 같은 mesh L2 domain에 들어간다.
- wlan0는 무선 링크만 담당하고 IP를 갖지 않는다.
- bat0가 애플리케이션이 보는 실제 통신 인터페이스다.
- node를 중간에 놓으면 BATMAN - adv가 자동으로 multi - hop relay 경로를 만든다.
- 카메라, 조종, ping은 모두 bat0 IP를 기준으로 통신한다.

확인 명령:

```
sudo batctl n
sudo batctl o
```

해석:

- batctl n: 직접 무선으로 보이는 neighbor
- batctl o: 목적지 originator별 selected nexthop
- batctl o에서 *가 붙은 줄이 현재 선택된 best path
- head 목적지의 nexthop이 node MAC이면 실제 relay 중이다.

3.3 Pi 3 Wi-Fi와 IBSS

Raspberry Pi 3 내장 Wi-Fi는 iw list에서 mesh point가 보이지 않고 IBSS만 지원했다. 따라서 현재 구성은 802.11s가 아니라 IBSS/ad-hoc 위에 BATMAN-adv를 올린 방식이다.

공통 설정:

```
MESH_IF=wlan0
BAT_IF=bat0
MESH_ID=HANSEL_MESH
MESH_FREQ=2437
IBSS_BSSID=02:12:34:56:78:9a
WIFI_POWER_SAVE=off
```

IBSS_BSSID를 고정한 이유는 같은 ESSID라도 서로 다른 IBSS cell로 갈라지는 문제를 줄이기 위해서다.

3.4 부팅 자동 연결

scripts/enable_mesh_autostart.sh는 systemd template service를 설치하고 role별 instance를 켜다.

예:

```
sudo ./scripts/enable_mesh_autostart.sh base
sudo systemctl enable hansel-mesh@base
```

부팅 후 실행 흐름:

```
systemd hansel-mesh@role
-> scripts/start_role_network.sh role
-> scripts/start_mesh.sh configs/role.env
-> base면 setup_base_gateway.sh
-> head/node면 setup_mesh_route_to_laptop.sh
```

따라서 현장에서는 Pi 전원을 켜 뒤 base와 노트북만 유선 연결하면 mesh가 자동으로 올라오는 구조다.

4. 데이터 통신 구조

4.1 조종 명령

조종 명령은 노트북에서 각 유닛의 mesh IP로 직접 UDP JSON을 보낸다. base가 명령을 받아서 다시 보내는 구조가 아니다.

```
laptop controller
-> UDP JSON 7000
-> base kernel routing
-> BATMAN-adv mesh
-> head/node mesh_control_server
```

패킷 예:

```
{
  "seq": 12,
  "target": "head",
  "command": "forward",
}
```

```

"source": "operator",
"time": 1780400166.3324077,
"speed": 1.0
}

```

UDP를 쓰는 이유:

- 조종은 최신 명령이 중요하다.
- 오래된 명령이 지연되어 도착하는 것보다 timeout으로 멈추는 편이 안전하다.
- live mode가 일정 간격으로 명령을 반복 송신한다.
- mesh_control_server.py는 일정 시간 명령이 없으면 watchdog stop을 건다.

4.2 카메라 영상

head의 카메라 영상은 RTP/H.264 over UDP로 노트북에 간다.

```

rpicam-vid
-> H.264 bitstream
-> GStreamer 또는 ffmpeg RTP packetizer
-> UDP 5600
-> laptop video_probe.py / receive_camera_stream.sh

```

기본값:

```

CAMERA_TRANSPORT=rtp
DEST_IP=192.168.60.2
DEST_PORT=5600

```

RTP를 쓰는 이유:

- RTP는 UDP 위에 sequence number, timestamp, payload type을 얹는다.
- raw UDP보다 수신 쪽에서 영상 패킷 순서와 손실을 판단하기 쉽다.
- 조종 명령 UDP 7000과 영상 UDP 5600은 서로 다른 포트라 충돌하지 않는다.

카메라 profile:

profile	의미	설정
0 / high	고화질	640x480, 15fps, 1.2Mbps
1 / medium	중간	480x360, 12fps, 0.8Mbps
2 / low	저화질	320x240, 10fps, 0.6Mbps
3 / survival	생존 모드	320x240, 8fps, 0.4Mbps

자동 품질 제어가 켜져 있으면 WARN/DANGER 상태에 따라 head에 camera_profile 명령을 보내고, head는 restart_camera_profile.sh로 스트림을 재시작한다.

5. 자동분리 알고리즘

5.1 왜 영상 품질을 1순위로 보는가

원격 조종 로봇에서는 ping이나 조종 명령이 살아 있어도, 카메라 화면이 깨지면 실제로 운전할 수 없다. UDP 조종 패킷은 크기가 작아서 멀리 가도 살아남을 수 있지만, H.264 영상은 대역폭과 연속 패킷 품질이 더 필요하다.

따라서 자동분리 기준은 다음 순서로 본다.

1. 수신 영상 품질: FPS, decode error, drop, stale
2. end - to - end ping: RTT, packet loss

3. 선택적 BATMAN TQ: batctl o의 selected TQ

5.2 품질 측정 값

monitor/video_probe.py가 노트북에서 카메라를 수신하면서 video_quality.jsonl에 샘플을 기록한다.

주요 필드:

값	의미
fps	실제 수신/디코딩 FPS
fps_ratio	실제 FPS / 목표 FPS
err_rate	H.264 decode error per second
drop_rate	drop frame per second
bitrate_kbps	수신 bitrate 추정
ts	샘플 timestamp

controller/quality_supervisor.py는 이 로그를 읽고 상태를 판단한다.

5.3 현재 임계값

WARN 조건:

기준	값
FPS ratio	< 0.85
decode error	> 0.20 / sec
drop frame	> 1.00 / sec
video stale	>= 1.0 sec
ping loss	> 5%
RTT	> 120 ms
BATMAN TQ	< 200

DANGER 조건:

기준	값
FPS ratio	< 0.60
decode error	> 1.00 / sec
drop frame	> 3.00 / sec
video stale	>= 2.0 sec
ping loss	> 10%
RTT	> 180 ms
BATMAN TQ	< 180

상태 유지 시간:

상태	유지 시간
WARN	3.0초 이상 지속
DANGER	1.5초 이상 지속

즉 순간적인 팀은 바로 분리하지 않고, 나쁜 상태가 일정 시간 지속될 때만 최종 상태가 바뀐다. 이 hysteresis가 없으면 통신이 잠깐 흔들릴 때 불필요하게 분리될 수 있다.

5.4 자동분리 동작

mesh_control_client.py 실행 시 --auto-detach 옵션이 있어야 실제 분리가 발생한다.

동작 순서:

```
video_probe가 영상 품질 로그 생성
-> quality_supervisor가 GOOD/WARN/DANGER 판단
-> WARN이면 속도 제한 + camera profile 낮춤
-> DANGER면 stop 전송
-> --auto-detach가 켜져 있으면 detach_order 순서로 분리
```

기본 분리 순서:

```
node2 -> node1
```

한 번 분리한 뒤에는 최소 detach_cooldown=6.0초를 기다린다.

중요한 안전 정책:

- detach 명령은 --target all에는 전송하지 않는다.
- 반드시 node1 또는 node2처럼 단일 target에만 보내도록 되어 있다.
- 자동분리 후 해당 node는 active moving targets에서 제거되어 더 이상 같이 움직이지 않는다.

6. 조종 로직

6.1 키보드 명령 분배

controller/mesh_control_client.py의 기본 target은 all이다.

키	head	node1/node2
w	forward	forward
s	backward	backward
a	left spin	stop
d	right spin	stop
q	forward_left	slow_forward
e	forward_right	slow_forward
z	backward_left	slow_backward
c	backward_right	slow_backward
x 또는 space	stop	stop
u/j/k	head servo up/down/center	전송 안 함
f/v	head front motor forward/stop	전송 안 함

설계 의도:

- head가 방향을 만들고 node들은 그 경로를 따라간다.
- node가 따로 조향하면 관절형 구조에서 꼬일 수 있으므로 node 조향 명령은 slow/stop으로 정규화한다.
- w만 누르면 모든 유닛이 같은 목표 CPS로 최대속도를 낸다.

6.2 서버 수신부

robot/mesh_control_server.py는 각 유닛에서 UDP 7000을 listen한다.

수신 후:

1. JSON parsing
2. command 추출
3. camera profile 명령이면 head에서 카메라 재시작
4. 그 외 명령은 motor_driver.py로 전달
5. 일정 시간 명령이 없으면 watchdog stop

7. 구동부 구조

7.1 GPIO pin map

기본 주행 DC 모터:

기능	GPIO
left ENA	18
left IN1	23
left IN2	24
right ENB	13
right IN3	27
right IN4	22

엔코더:

기능	GPIO
left A	20
left B	21
right A	16
right B	26

head 앞쪽 고개 유닛 DC 모터:

기능	GPIO
front left ENA	12
front left IN1	3
front left IN2	8
front right ENB	19
front right IN3	5
front right IN4	7

서보:

기능	GPIO
detach servo	6

기능	GPIO
head servo	17

코드는 시작 시 GPIO 중복을 검사한다. 같은 GPIO가 두 기능에 동시에 들어가면 GPIO pin conflict로 실행을 막는다.

7.2 Role config

role_config(role)이 head/node별 설정을 만든다.

head:

- 주행 모터 2개
- 엔코더 2쌍
- detach servo
- head servo
- front motor 2개

node1/node2:

- 주행 모터 2개
- 엔코더 2쌍
- detach servo
- head servo 없음
- front motor 없음

8. PID 제어 로직

8.1 목표

각 바퀴의 엔코더 count per second를 목표 CPS에 맞춘다. 단순 PWM 고정이 아니라 엔코더 feedback으로 왼쪽/오른쪽 속도 차이를 줄인다.

기본 목표 속도:

```
full_speed_cps_left = 800
full_speed_cps_right = 800
speed scale = 0.0 ~ 1.0
target_cps = full_speed_cps * speed scale * command ratio
```

8.2 엔코더 처리

motor_driver.py는 별도 thread에서 encoder A/B 핀을 polling한다.

흐름:

```
GPIO input A/B
-> quadrature state 읽기
-> state transition table로 count 증가/감소
-> control loop에서 dt 기준 cps 계산
```

RPi.GPIO event detect 대신 polling thread를 쓴 이유는 OS/kernel 조합에 따라 edge callback이 불안정할 수 있기 때문이다.

8.3 PID 계산

제어 loop는 기본 CONTROL_INTERVAL=0.05초마다 돈다.

계산:

```
measured_cps = abs(delta_encoder_count) / dt
error = target_cps - measured_cps
integral = clamp(integral + error * dt)
derivative = (error - prev_error) / dt
```

PWM은 feed-forward와 PID output을 합쳐 만든다.

```
feed_forward = MIN_PWM + target_cps / max_cps * (MAX_PWM - MIN_PWM)
pid_output = kp * error + ki * integral + kd * derivative
requested_pwm = feed_forward + pid_output
```

기본값:

```
MIN_PWM = 25
MAX_PWM = 100
PWM_RAMP_PER_SEC = 220
kp_left = 0.035
ki_left = 0.015
kd_left = 0.0
kp_right = 0.035
ki_right = 0.015
kd_right = 0.0
```

8.4 PWM ramp

요청 PWM을 바로 넣지 않고 초당 변화량을 제한한다.

```
max_delta = PWM_RAMP_PER_SEC * dt
pwm = clamp(requested_pwm, previous_pwm - max_delta, previous_pwm + max_delta)
```

효과:

- 출발 시 모터가 갑자기 튀는 문제 감소
- 배터리 순간 전류 부담 감소
- 좌우 속도 안정화

8.5 head front motor 동기화

head 앞쪽 고개 유닛에 달린 DC 모터 2개는 뒤쪽 주행 바퀴와 같은 명령을 따라간다.

```
front_follow_drive = True
front_pwm = rear_pid_pwm * front_speed_ratio
```

조향 시에도 왼쪽/오른쪽 방향과 PWM을 각각 따라가므로 head의 앞쪽 바퀴도 뒤쪽 바퀴와 같은 방식으로 움직인다.

9. 서보와 PWM

9.1 pigpio를 쓰는 이유

서보는 일반적으로 50Hz, 약 20ms frame 안에 0.5ms~2.5ms 폭의 pulse를 넣어 각도를 만든다. RPi.GPIO의 일반 PWM은 Linux scheduler 지연 영향을 크게 받기 때문에 서보가 튀거나 지연될 수 있다.

pigpio는 daemon이 GPIO pulse를 더 정밀하게 관리한다.

장점:

- 일반 GPIO 핀에서도 서보 pulse 생성 가능

- hardware - timed 방식이라 Python sleep보다 지터가 작다.
- head servo와 detach servo 모두 같은 방식으로 제어 가능

현재 코드:

- pigpio module과 daemon이 있으면 PiggpioServoPwm
- pigpio가 없으면 SoftwareServoPwm fallback

9.2 head servo

기본값:

```
HEAD_SERVO_MIN_ANGLE = 20
HEAD_SERVO_MAX_ANGLE = 150
HEAD_SERVO_CENTER_ANGLE = 70
HEAD_SERVO_STEP_ANGLE = 1
HEAD_SERVO_RAMP_STEP_ANGLE = 1.0
HEAD_SERVO_RAMP_INTERVAL = 0.06
HEAD_SERVO_MIN_PULSE_US = 600
HEAD_SERVO_MAX_PULSE_US = 2400
```

갑자기 확 올라가는 문제를 줄이기 위해 즉시 목표 각도로 보내지 않고 ramp loop가 조금씩 목표 각도로 이동한다.

현재 각도 -> 목표 각도까지
0.06초마다 1도씩 이동

9.3 detach servo

기본값:

```
DETACH_REST_ANGLE = 20
DETACH_PRESS_ANGLE = 75
DETACH_PRESS_TIME = 0.35초
```

동작:

```
detach_press
-> press angle 이동
-> 0.35초 유지
-> rest angle 복귀
```

자동분리도 결국 이 detach_press 명령을 해당 node에 UDP로 보내는 방식이다.

9.4 SoftwareServoPwm fallback

pigpio가 없을 때도 테스트를 멈추지 않기 위해 SoftwareServoPwm이 있다.

동작:

```
20ms frame 시작
-> GPIO HIGH
-> pulse_us만큼 대기
-> GPIO LOW
-> 다음 frame까지 대기
```

마지막 수백 us는 busy wait로 지터를 줄인다.

환경변수:

```
HANSEL_SERVO_SOFT_PWM_FRAME_US=20000
HANSEL_SERVO_SOFT_PWM_SPIN_US=300
```

단, fallback은 어디까지나 비상용이다. 발표에서는 pigpio가 기본 설계라고 설명하는 것이 맞다.

10. monitor 코드 설명

10.1 video_probe.py

노트북에서 실행한다. 카메라 수신과 품질 측정을 동시에 한다.

내부 동작:

```
ffmpeg 실행
-> RTP/H.264 또는 raw UDP 수신
-> 화면 표시 또는 null output
-> -progress stdout parsing
-> stderr decode error counting
-> video_quality.jsonl 기록
-> 선택적으로 dashboard UDP 7100 전송
```

자동분리의 핵심 센서다.

10.2 quality_supervisor.py

video_quality.jsonl을 읽고 품질 상태를 판단한다.

상태:

- GOOD: 정상
- TRANSIENT: 나쁜 상태가 잠깐 발생했지만 유지 시간이 부족함
- WARN: 감속과 카메라 profile 하향
- DANGER: stop과 자동분리 후보

10.3 metrics_agent.py

각 Pi에서 실행한다. dashboard 시각화용 데이터를 만든다.

수집:

- iw dev wlan0 station dump: RSSI, bitrate
- batctl n: direct neighbor
- batctl o: TQ, selected nexthop
- ip neigh show dev bat0: MAC - IP 매핑
- ping: end - to - end RTT/loss

10.4 dashboard.py

노트북에서 실행한다.

```
UDP 7100 collector
HTTP 8080 dashboard
```

보여주는 것:

- node online 상태
- 직접 링크 RSSI
- BATMAN TQ
- end - to - end RTT
- 영상 FPS/error/drop
- 통신 품질과 영상 품질의 상관관계

인터넷이 없으면 Chart.js/vis - network CDN이 안 뜰 수 있으므로, monitor/web/index.html에는 기본 fallback 화면이 들어 있다.

11. 실행 절차

11.1 코드 배포

노트북에서 base로:

```
cd ~/Projects/HANSEL_MESH
scp -r configs controller docs monitor robot scripts services README.md \
hansel@192.168.60.1:/home/hansel/HANSEL_MESH/
```

base에서 head/node로:

```
cd ~/HANSEL_MESH
./scripts/deploy_from_base.sh
```

11.2 mesh 확인

노트북:

```
ping -c 3 192.168.60.1
ping -c 3 192.168.50.10
ping -c 3 192.168.50.11
ping -c 3 192.168.50.12
```

base:

```
sudo batctl n
sudo batctl o
```

11.3 서버 실행

base에서 원격 실행:

```
ssh -t hansel@192.168.50.10 'cd ~/HANSEL_MESH && sudo python3 robot/mesh_control_server.py -
-role head'
ssh -t hansel@192.168.50.11 'cd ~/HANSEL_MESH && sudo python3 robot/mesh_control_server.py -
-role node1'
ssh -t hansel@192.168.50.12 'cd ~/HANSEL_MESH && sudo python3 robot/mesh_control_server.py -
-role node2'
```

head 기대 로그:

```
[head] pigpio connected; using hardware-timed PWM for detach servo
[head] pigpio connected; using hardware-timed PWM for head servo
```

11.4 카메라 시작

head:

```
cd ~/HANSEL_MESH
CAMERA_TRANSPORT=rtp bash scripts/restart_camera_profile.sh 1 192.168.60.2 5600
```

노트북:

```
cd ~/Projects/HANSEL_MESH
python3 monitor/video_probe.py --transport rtp --port 5600 --target-fps 12 \
--log video_quality.jsonl
```

11.5 자동분리 포함 조종

노트북:

```
cd ~/Projects/HANSEL_MESH
python3 controller/mesh_control_client.py --target all --speed 1 --live \
--quality-log video_quality.jsonl \
--quality-target-fps 12 \
--quality-warn-speed 0.35 \
--auto-camera-profile \
--camera-transport rtp \
--auto-detach \
--detach-order node2,node1
```

--auto-detach를 빼면 자동분리 없이 감속, stop, camera profile 변경만 확인할 수 있다.

12. 발표에서 강조할 점

1. AP/client 방식이 아니라 모든 Pi가 같은 mesh에 참여한다.
2. base는 애플리케이션 relay가 아니다. kernel routing + BATMAN - adv만 담당한다.
3. node들은 별도 프로그램으로 데이터를 중계하지 않아도 Layer 2에서 frame을 relay한다.
4. 원격 조종의 핵심은 ping이 아니라 영상 품질이다.
5. 자동분리 기준은 FPS, decode error, drop, stale을 1순위로 둔다.
6. PID는 엔코더 CPS를 기준으로 좌우 바퀴 속도를 맞춘다.
7. head만 조향하고 node는 직진 추종한다.
8. 서보는 pigpio hardware - timed PWM으로 일반 GPIO에서도 안정적인 pulse를 만든다.
9. monitor는 자동분리의 입력값과 발표 시각화를 동시에 제공한다.

13. 한계와 다음 단계

현재 한계:

- 동글 없이 같은 wlan0로 mesh와 조난자 휴대폰 AP를 동시에 제공하기 어렵다.
- 영상 품질 기준은 실험값 기반으로 조정이 필요하다.
- Pi만으로 servo PWM을 만들 수 있지만, 장기적으로는 전용 MCU가 더 안정적이다.

다음 단계:

- Arduino Nano 또는 RP2040/Pico에 servo PWM을 맡기기
- 자동분리 기준을 현장 로그 기반으로 재튜닝
- dashboard 로그를 발표 그래프로 정리
- 조난자 휴대폰 접속용 별도 AP 장치 또는 외장 Wi-Fi 동글 추가 검토
- node3 추가 시 TARGETS, configs/node3.env, detach-order node3,node2,node1로 확장

14. 빠른 Q&A

Q. 왜 BATMAN을 쓰는가? A. node를 중간에 놓았을 때 사용자가 route를 직접 바꾸지 않아도 Layer 2에서 next-hop을 자동 선택하기 위해서다.

Q. 카메라가 끊겼는데 조종은 되는 이유는? A. 조종 UDP packet은 매우 작고, 영상은 연속적인 고대역폭 packet이 필요하기 때문이다. 그래서 자동분리 기준은 영상 품질을 우선한다.

Q. 자동분리는 언제 발생하는가? A. --auto-detach를 켜 상태에서 DANGER가 1.5초 이상 지속되면 stop 후 node2 -> node1 순서로 detach한다.

Q. pigpio가 꼭 필요한가? A. 주행만 하면 필수는 아니지만, head servo와 detach servo를 안정적으로 움직이려면 필요하다. fallback은 있지만 발표에서는 pigpio를 기본 설계로 보면 된다.

Q. PID는 무엇을 제어하는가? A. PWM 자체가 아니라 엔코더로 측정한 count per second가 목표 CPS에 가까워지도록 PWM을 조절한다.